

Databases & SQL

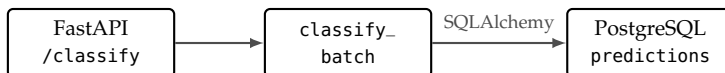
2026-05-10 · happy pear Nachtigall

IN COMPARISON, DUMBLEDORE'S PENSIEVE IS A JOKE.

The Why

We have a service that classifies, but it does not remember anything. The moment the response leaves the server, the prediction is gone. This makes it hard to track the model's performance over time, compare versions, or build any type of features and functionality that revolve around past predictions.

WE NEED TO WIRE THE API ONTO A DATABASE. This block is the first that adds persistent state. Every later block depends on the database schema you design today.



WHY PERSIST AT ALL? A response that vanishes the moment it is sent is a response that cannot be counted, compared, or queried. A database makes the prediction permanent: Every drawing becomes a row, the class label becomes a column, the confidence becomes another, and the model version becomes a third.

BEYOND THIS COURSE, the same table is where production teams build retraining datasets, let users correct wrong predictions for labelled data, and detect distribution drift. We do not do any of those here, but it is trivial to see how every one of them starts with having the database.

SQLALCHEMY. The Object-Relational Mapper, abbreviated ORM, that bridges Python objects and relational tables. You write Python classes; SQLAlchemy generates the Structured Query Language, abbreviated SQL, behind them, so the handler talks to PostgreSQL without composing query strings by hand.

<https://www.sqlalchemy.org/>

Figure 1: The persistence flow. POST /classify computes a prediction with `classify_batch`, the handler maps the result through the `Prediction` SQLAlchemy model, and a row lands in the `predictions` table in PostgreSQL before the response goes out.

Hands On Experience

CONSIDER THIS SCENARIO: A teammate asks how confident the model has been on average across the last hundred predictions. You realise nobody can answer that question. The API returned the confidence to every caller, but nothing recorded it. The only way back to those numbers is to ask the users to draw their digits again, which is not an option. In an increasingly data-driven world, data defines your product, your moat, and your competitive advantage. Not having the data is like throwing away your most valuable asset, and until someone comes up with a solution for time-travel, it's a non-recoverable loss.

THE SCHEMA IS A CONTRACT, the same way the API contract is a contract. Get it right and every later question, distribution per digit, average confidence per model version, error rate over time, becomes a query. Get it wrong and you find out later, with a hundred thousand rows already written, that the column you need does not exist. This block introduces migrations as the tool that lets the schema evolve without losing the rows already in it.

THIS SIXTH BLOCK introduces persistence for PixelWise. By the end you will have

- written your first SQL queries against a PostgreSQL database,
- modelled the predictions table as a SQLAlchemy class,
- wired POST /classify so every prediction becomes a row,
- replaced the GET /results stub with a real query,
- run your first schema migration with Alembic to add a missing column,
- and configured database credentials through .env.

DESIGNING SCHEMAS is its own discipline. We deliberately start with a near-trivial schema, three columns, and grow it through a migration in the same session. The point is to feel the cycle of design, deploy, regret, migrate, which is how every real database evolves.

Optional Exercise: Catch Up to v0.4

CATCHING UP VIA TAG. This catch-up extends the one in Block 5. If you have not done that one, do it first: It lands you at v0.3, the end-of-Block 4 state. The exercise here picks up from there and rewinds to v0.4, the end-of-Block 5 state, with the FastAPI service exposing `POST /classify` and `GET /health`, API key authentication, basic rate limiting, and a systemd-supervised uvicorn running on prod.

BOTH VMs THIS TIME. Block 5 ended with the service actually running on prod, so unlike the previous catch-up, this one brings both machines up to v0.4. The Git-tracked changes since v0.3 include `app/main.py`, `deploy/pixelwise.service`, an extended `setup-server.sh` that installs the systemd unit on prod, an updated `requirements.txt` adding `fastapi`, `uvicorn`, `slowapi`, and `pydantic`, and a refreshed `.env.example` with `SECRET_API_KEY`. The local `.env` on prod needs the new key written by hand; `setup-server.sh` then pulls the model, copies the unit, enables, and restarts the service in one shot.

RUN THE CATCH-UP ON dev. Rewind the repository to v0.4, refresh `.env` from the updated example and edit it to set `SECRET_API_KEY`, re-install pinned dependencies if your virtual environment was wiped, then run `setup-server.sh` to pull the model artefact. The exact commands are in the margin.

RUN THE CATCH-UP ON prod. Same sequence on prod, where `setup-server.sh` also copies the systemd unit into place and restarts the service. After the script finishes, verify the API is reachable from dev.

YOU ARE NOW at the state where Block 5 ended, with both machines aligned, ready to start Block 6.

TAG MAP. v0.4 marks the end of Block 5, v0.5 will mark the end of this block. Browse the full set at <https://github.com/schutera/pixelwise/tags>.

SEQUENCE ON dev.

```
cd ~/pixelwise
git fetch origin --tags
git reset --hard v0.4
cp .env.example .env
nano .env
source .venv/bin/activate
pip install -r requirements.txt
bash setup-server.sh
```

Edit `.env` to set `SECRET_API_KEY` to any string of your choosing; `openssl rand -hex 32` prints a clean random value if you want one. If you already have a `.env` from earlier work, skip the `cp` and just edit the file in place. If `.venv/` is missing, run `python3 -m venv .venv` before the `source` step.

SEQUENCE ON prod.

```
ssh produser@192.168.56.11
cd /opt/pixelwise
git fetch origin --tags
git reset --hard v0.4
cp .env.example .env
nano .env
source .venv/bin/activate
pip install -r requirements.txt
bash setup-server.sh
```

Use the same `SECRET_API_KEY` value as on dev. If you already have a `.env` on prod, skip the `cp` and edit the file in place. Then from dev: `curl http://192.168.56.11:8000/health` should return `{"status": "ok" ...}`.

Relational Databases and SQL

A RELATIONAL DATABASE STORES DATA IN TABLES. Each table has a fixed schema: a list of named columns, each with a type. Each row is a single record. The schema is the contract that every row obeys; the database refuses inserts that violate it.

SQL IS THE LANGUAGE you use to talk to a relational database. Four operations cover almost everything:

```
-- Insert a row
INSERT INTO predictions (prediction, confidence,
                        model_version)
VALUES ('7', 0.94, 'v1');

-- Read rows
SELECT prediction, confidence FROM predictions
    WHERE model_version = 'v1'
    ORDER BY created_at DESC
    LIMIT 10;

-- Update a row
UPDATE predictions SET confidence = 0.95
    WHERE id = 42;

-- Delete a row
DELETE FROM predictions WHERE id = 42;
```

THE FOUR OPERATIONS ARE WHAT EVERY WEB APPLICATION DOES TO ITS DATABASE ALL DAY LONG: write a row when something happens, read rows back when somebody asks, update a row when state changes, delete one when it should be gone. Every higher-level pattern, an ORM call, a migration, an analytics query, reduces to a sequence of these.

PostgreSQL over SQLite

SQLITE IS CONVENIENT. The database is a single file, there is no server to install, no users, no network configuration. For a single-developer prototype or a desktop application, this is exactly what you want.

VOCABULARY TO KEEP STRAIGHT. A *table* is a named set of rows; a *row* is one record; a *column* is a named field with a type; a *primary key* is the column (or set of columns) that uniquely identifies each row. Most tables use an auto-incrementing integer or a UUID as their key.

WHAT YOU WILL MEET LATER. JOIN combines rows from multiple tables; GROUP BY, COUNT, and AVG aggregate; ORDER BY and LIMIT shape the output; CREATE INDEX speeds up queries that scan large tables. The full treatment belongs in a dedicated database course; the four operations above are enough to get PixelWise running.

WHERE SQLITE STOPS WORKING. SQLite has no user authentication, no password protection, and limited concurrency: a single writer blocks all readers. For a multi-user web service where 30 students hit the same database at the same second, the file-lock model becomes the bottleneck. The same file is also harder to back up while the application is running.

POSTGRESQL IS A PRODUCTION DATABASE. It runs as a separate process, accepts connections over the network, has real users with real permissions, and supports concurrent reads and writes without serialising the whole file. `psql` is the operator's interface, the same shape of tool as `systemctl` from Block 5.

SETTING UP THE DATABASE on the server is two commands inside `psql`. You log in as the `postgres` superuser, create a dedicated `pixelwise` user with a password, then create a `pixelwise` database owned by that user, and quit with `\q`. The exercise below has the exact statements `PixelWise` needs.

CONNECT AS THE APPLICATION USER to confirm the credentials work before any Python is involved. The prompt that comes back from `psql` is the same one your service will see when it connects through `SQLAlchemy` in the next section. If `psql` cannot reach the database, neither can the API; debug here first. The exercise below runs the connection command end to end.

PostgreSQL: <https://www.postgresql.org/>

PostgreSQL Docs: <https://www.postgresql.org/docs/current/>

SQLite: <https://www.sqlite.org/>

WHY A DEDICATED USER? The `postgres` superuser can drop any database, create any user, and read any table. A dedicated `pixelwise` user can only do what its privileges allow, which by default is read and write its own tables. The principle is the same as the non-root producer from Block 1: the account the application uses is the smallest one that works.

Exercise: PostgreSQL on the Server

INSTALL POSTGRESQL ON prod. With the theory above as the why, the how is two commands. On the prod VM, install the server and start it:

```
ssh produser@192.168.56.11
sudo apt update
sudo apt install -y postgresql
sudo systemctl status postgresql
```

SET THE DATABASE PASSWORD IN .env. The password is a secret, so it lives in .env on each VM and never in version control. Add the variable on prod, plus a placeholder in .env.example on dev so the contract is documented while the value stays local:

```
# on prod
echo "DB_PASSWORD=secret" >> /opt/pixelwise/.env
```

```
# on dev, then commit and push
echo "DB_PASSWORD=" >> .env.example
```

CREATE THE ROLE AND DATABASE. Source .env so the password is available as a shell variable, then create the role and database in two psql -c calls:

```
set -a; source /opt/pixelwise/.env; set +a
sudo -u postgres psql -c \
    "CREATE USER pixelwise \
    WITH PASSWORD '$DB_PASSWORD';"
sudo -u postgres psql -c \
    "CREATE DATABASE pixelwise OWNER pixelwise;"
```

VERIFY THE CONNECTION with the application user:

```
psql -U pixelwise -d pixelwise -h localhost
```

If psql fails to connect, fix the credentials before going any further; SQLAlchemy will report the same error in a less direct way. A working psql is the foundation of everything the rest of the block builds on.

TEACH setup-server.sh TO PROVISION THE DATABASE. Add postgresql to the apt install line, then append a prod-only stanza next to the systemd block from Block 5. Open the file in nano on dev:

WHY set -a? set -a tells bash to export every variable defined from now on. source .env reads the file as shell commands, so the assignments happen in the current shell. set +a turns the auto-export back off. The same pattern lets a script load .env before running anything that depends on it.

```
nano setup-server.sh
```

```
# Provision the pixelwise role and database on prod
if command -v psql >/dev/null 2>&1 && \
  id produser >/dev/null 2>&1 && \
  [ -f /opt/pixelwise/.env ]; then
  set -a; source /opt/pixelwise/.env; set +a
  sudo -u postgres psql -tAc \
    "SELECT 1 FROM pg_roles WHERE rolname='pixelwise'" \
    | grep -q 1 || \
  sudo -u postgres psql -c \
    "CREATE USER pixelwise \
    WITH PASSWORD '$DB_PASSWORD';"
  sudo -u postgres psql -tAc \
    "SELECT 1 FROM pg_database WHERE datname='pixelwise'" \
    | grep -q 1 || \
  sudo -u postgres createdb -O pixelwise pixelwise
fi
```

Idempotent by design. Each `psql -tAc` probe asks PostgreSQL whether the role or database already exists; the `||` short-circuits the create when the answer is yes. Run the script ten times, the role and database are still there exactly once. The same `id produser` guard from Block 5 keeps the stanza from running on dev, where there is no PostgreSQL server.

Commit, push, and on prod run `bash setup-server.sh` to verify the stanza is idempotent. The second run should change nothing because the role and database already exist.

SQLAlchemy: Tables as Python Classes

SQLALCHEMY IS A PYTHON LIBRARY that maps database tables to Python classes. You define a Prediction class with fields like prediction, model_version, and created_at; SQLAlchemy generates the SQL to create the table, insert rows, and query data. This is called an ORM, an object-relational mapper, and is standard in Python web development.

THE MODEL CLASS declares the columns and their types in plain Python. A Prediction class inherits from a SQLAlchemy declarative base, sets __tablename__ to predictions, and lists each column as a class attribute: An auto-incrementing integer primary key, a non-null string for the predicted class, a non-null string for the model version, and a timestamp that defaults to the current UTC time. The exercise below has the full app/models.py the API will import.

AN ENGINE AND A SESSION are the runtime objects that connect the model to the database. create_engine reads the DATABASE_URL from the environment and opens a pool of connections; sessionmaker returns a SessionLocal factory the handlers will call once per request; Base.metadata.create_all(engine) creates the table on first run. The exercise wires these three lines into a small initialisation script the team runs on prod before the first deploy.

INSERTING A ROW from inside the API handler is now a few lines: Open a session from SessionLocal, construct a Prediction with the fields from the classifier result, add it to the session, and commit. The handler still calls classify_batch from Block 4 and returns ClassifyResponse from Block 5; the only addition is one round-trip to the database before the response leaves the server. The exercise below wires it up inside POST /classify.

SQLAlchemy: <https://www.sqlalchemy.org/>
 SQLAlchemy Tutorial:
<https://docs.sqlalchemy.org/en/20/tutorial/>

WHY AN ORM? Two reasons. The first is ergonomic: writing Python is faster than writing string SQL, and the editor knows the field names. The second is safety: SQLAlchemy uses parameterised queries by default, which means user input never ends up concatenated into a SQL statement. SQL injection becomes a footgun the framework refuses to hand you.

THE CONNECTION STRING is a single URL:

postgresql://user:pass@host/dbname.
 It encodes everything the driver needs to reach the database, and it lives in .env for the same reasons the API key did in Block 5. A leaked DATABASE_URL is a leaked database.

Exercise: The Prediction Model

TURN THE THEORY ABOVE INTO CODE. On dev, install the driver and the ORM into the virtual environment from the repository root:

```
cd ~/pixelwise
source .venv/bin/activate
pip install sqlalchemy psycopg2-binary
pip freeze > requirements.txt
```

WRITE `app/models.py` with the minimal schema, only the class label and the model version. From the repository root on dev:

```
cd ~/pixelwise
nano app/models.py
```

Paste the following, save with `Ctrl+O`, exit with `Ctrl+X`:

```
from sqlalchemy import (Column, Integer, String,
                        DateTime)
from sqlalchemy.orm import declarative_base
from datetime import datetime

Base = declarative_base()

class Prediction(Base):
    __tablename__ = "predictions"
    id = Column(Integer, primary_key=True)
    prediction = Column(String, nullable=False)
    model_version = Column(String, nullable=False)
    created_at = Column(DateTime,
                        default=datetime.utcnow())
```

ADD `DATABASE_URL` to `~/pixelwise/.env` and `~/pixelwise/.env.example` on dev. The URL interpolates `DB_PASSWORD` from the same file, so the secret lives in one place:

```
DATABASE_URL=postgresql://pixelwise:${DB_PASSWORD}@localhost/pixelwise
```

THE SCHEMA IS INTENTIONALLY TOO SMALL. The first version stores only the predicted class and the model version. Confidence is missing on purpose; you will feel the gap in the next exercise and use Alembic to fix it. The point is to feel the migration cycle, not to skip past it.

CREATE THE TABLE ONCE from a small initialisation script saved as `init_db.py` at the repository root on dev:

```
cd ~/pixelwise
nano init_db.py
```

Paste the following:

```
from sqlalchemy import create_engine
from app.models import Base
import os
from dotenv import load_dotenv

load_dotenv()
engine = create_engine(os.getenv("DATABASE_URL"))
Base.metadata.create_all(engine)
```

Commit and push from dev, then run the script on prod where the database lives:

```
# on prod, after git pull
cd /opt/pixelwise
source .venv/bin/activate
python init_db.py
```

Confirm with `psql` that `\dt` lists the predictions table.

Exercise: Wiring the API to the Database

UPDATE `app/main.py` so `POST /classify` writes a row before returning the response. Reopen the file from the repository root on dev:

```
cd ~/pixelwise
nano app/main.py
```

The handler from Block 5 stays the same shape; one new dependency creates a database session per request:

```
from app.models import Prediction, SessionLocal

@app.post("/classify",
          response_model=ClassifyResponse,
          dependencies=[Depends(verify_api_key)])
def classify(req: ClassifyRequest):
    arr = np.array(req.pixels,
                   dtype=np.uint8)[np.newaxis]
    result = classify_batch(arr)[0]

    db = SessionLocal()
    db.add(Prediction(
        prediction=result["prediction"],
        model_version="v1"))
    db.commit()
    db.close()

    return result
```

REPLACE THE `GET /results` STUB in the same `app/main.py` with a real query:

```
@app.get("/results")
def results():
    db = SessionLocal()
    rows = (db.query(Prediction)
            .order_by(Prediction.created_at.desc())
            .limit(20).all())
    db.close()
    return {"results": [
        {"id": r.id,
         "prediction": r.prediction,
```

ONE SESSION PER REQUEST. A session is the unit of database work. Open it at the start of the handler, close it at the end. The framework patterns FastAPI documents (`Depends` for sessions) do this more elegantly; the manual version above is the simplest one that shows what is happening.

```
"model_version": r.model_version,  
"created_at": r.created_at.isoformat()  
for r in rows]}
```

RESTART THE SERVICE AND FEED IT DATA. Twenty `curl` requests later, `GET /results` returns twenty rows. The stub from Block 5 is now complete; the system stores what it predicts.

ASK THE QUESTION the schema cannot answer: "how confident is the model on average?" The API returns confidence to every caller, but the database only stored the class label. You cannot answer the question from the data on disk. That is the gap the next exercise closes.

Schema Migrations with Alembic

SCHEMAS ALWAYS CHANGE. You design the table, deploy it, store 500 predictions. Then you realise you forgot a column. You cannot drop the table and recreate it; those 500 rows are real data. You need a tool that alters the live schema without losing them.

ALEMBIC is the migration tool for SQLAlchemy. It generates migration scripts from changes you made to the model, applies them in order, and tracks which migrations have run on which database. The cycle is three commands: `alembic init` once per repository to lay down the migrations folder, `alembic revision --autogenerate` after every model edit to draft the migration, and `alembic upgrade head` to apply it. The exercise below runs the full cycle to add a missing confidence column.

IN A REAL SYSTEM with millions of rows and live traffic, you cannot just drop and recreate. Migrations are how schemas evolve safely, in production, with the application running. This topic is covered in much more depth in dedicated database and backend courses; here we run one migration end to end so the cycle of edit-model, generate-migration, apply-migration is concrete.

Alembic: <https://alembic.sqlalchemy.org/>

THE METAPHOR. A migration is a recipe that describes a change to the schema and how to roll it back. The migration tool keeps an internal log of which migrations have run, applies new ones in order, and refuses to run the same one twice. The result is that the schema in the database always matches the migrations folder in Git.

THE AUTOGENERATE IS A DRAFT, NOT A FINAL ANSWER. Alembic compares the model in code against the schema in the database and guesses what changed. For straightforward additions it gets it right; for renames or splits it needs a human to read the generated script and adjust it. Always read the migration before you run it.

SIDENOTE: REDIS. Redis is an in-memory key-value store with sub-millisecond reads. It is the right tool for caching, session storage, and rate-limit counters. For PixelWise, every drawing is unique pixel data, so the cache hit rate would be effectively zero; we mention it so you recognise it later.

Redis: <https://redis.io/>

Exercise: First Migration with Alembic

INITIALISE ALEMBIC IN THE REPO. With the migration cycle from the theory above in mind, configure the tool to use the same engine as the application. alembic init writes its scaffold into the current directory, so run everything from the repository root on dev:

```
cd ~/pixelwise
source .venv/bin/activate
pip install alembic
pip freeze > requirements.txt
alembic init alembic
```

Edit alembic.ini so sqlalchemy.url reads from .env, and edit alembic/env.py to import Base from app.models so autogenerate sees your tables.

ADD THE MISSING COLUMN to the model. Reopen app/models.py from ~/pixelwise on dev:

```
cd ~/pixelwise
nano app/models.py

from sqlalchemy import Float

class Prediction(Base):
    __tablename__ = "predictions"
    id = Column(Integer, primary_key=True)
    prediction = Column(String, nullable=False)
    confidence = Column(Float)          # NEW
    model_version = Column(String, nullable=False)
    created_at = Column(DateTime,
                              default=datetime.utcnow)
```

GENERATE THE MIGRATION from the repository root on dev, commit the new file under alembic/versions/, and push:

```
cd ~/pixelwise
alembic revision --autogenerate \
    -m "add confidence column"
```

Apply the migration on prod, where the database lives:

```
# on prod, after git pull
cd /opt/pixelwise
source .venv/bin/activate
alembic upgrade head
```

READ THE GENERATED CONFIG.

alembic init alembic writes a folder of starter files. Skim them once before you change anything; the structure is small and the mental model becomes obvious by reading the templates.

READ THE MIGRATION BEFORE YOU RUN IT. The generated script lives in alembic/versions/. Open it, confirm the only operation is add_column("confidence"), and only then run upgrade head. A wrong migration on a populated table is much harder to undo than to prevent.

VERIFY ON prod WITH psql:

```
psql -U pixelwise -d pixelwise -h localhost \
-c "SELECT prediction, confidence
    FROM predictions
    ORDER BY created_at DESC LIMIT 5;"
```

Old rows show NULL for confidence; new predictions store the real number. That gap between old and new rows is the visible signature of a migration: the column exists everywhere, but its values are populated only from the moment the new code shipped.

WIRE CONFIDENCE INTO THE HANDLER in ~/pixelwise/app/main.py on dev:

```
db.add(Prediction(
    prediction=result["prediction"],
    confidence=result["confidence"],
    model_version="v1"))
```

Restart the service. Draw a digit; confirm the new row in psql has both fields populated. Commit, push, and the repository now matches v0.5:

```
cd ~/pixelwise
git add app/main.py app/models.py alembic/ \
    alembic.ini requirements.txt .env.example
git commit -m "Add PostgreSQL persistence and first migration"
git push
```

A NOTE ON WHAT COMES NEXT. The system is reachable on the network and remembers what it does, but nobody outside the SSH tunnel can use it. Making PixelWise accessible from a browser at a real URL is a thread we will revisit later in the course.

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thoughts during and after the exercises:

- Why do we store predictions in a database rather than just returning them in the API response?
- What is the difference between dropping and recreating a table versus running a migration?
- Why did the old rows show NULL for the confidence column after the migration?
- What happens if you store the database password directly in your Python code instead of in `.env`?
- Why is a dedicated database user with minimal privileges better than connecting as the superuser?
- How does the ORM make SQL injection harder, and where does the protection break down?

RECAP of key concepts:

- Relational databases store data in typed tables; SQL is the language that queries and changes them.
- PostgreSQL is the production database for this course; SQLite is too limited for a multi-user web service.
- SQLAlchemy maps tables to Python classes and uses parameterised queries by default.
- Migrations let schemas change without losing rows; Alembic generates and applies them.
- Database credentials live in `.env`, never in code; the application user has minimal privileges.

MILESTONE. Every digit prediction is stored in PostgreSQL with its confidence and model version. `GET /results` returns real data from the table; the Block 5 stub is gone. You have run your first schema migration on a live database with Alembic and watched old rows coexist with new ones. Making PixelWise reachable as a website from a real browser is a step we will take later in the course.

SECURITY THREAD. DB credentials live in `.env`, never in code. The dedicated `pixelwise` user has no superuser rights and cannot create databases. SQLAlchemy's parameterised queries prevent SQL injection by default; never build queries with string formatting.

TEASER. The service stores predictions and returns them, but only somebody with `curl` and the API key can ask. What does it take to put this in front of a real browser at a real URL?